Listing 5.11 Pact Builder, part 2

```
import au.com.dius.pact.consumer.dsl.DslPart
import au.com.dius.pact.consumer.dsl.PactDslJsonBody

    val body: DslPart = PactDslJsonArray.arrayEachLike()
        .integerType("id")      #1
        .stringType("name")     #1
        .integerType("year")    #1
        .closeObject()!!
        ...
        .willRespondWith()
        .status(200)
        .headers(mapOf("Content-type" to "application/json"))
        .body(body)
```

#1 Pact DSL matching methods

The following method chained to the Pact contract builder is `willRespondWith`, which starts the response section of the consumer interaction. The first part of the response section sets the HTTP status code with a number as an argument, for example, 200. Then, the `headers` method takes a map of two string elements. The header in this example sets the content type to JSON for the body of the response object. The `body` method supports different types of objects as arguments, such as `JSON` or `Document`. In this example, we're using the `DslPart` object type.

The body is constructed using the Pact DSL to build the JSON body, including Pact matchers. Matchers provide the ability to define the values of the JSON as the type of the value rather than the exact value itself. The movie's response contains an array of randomly generated objects with the ID, name, and year of the movie. Each part of the response is chained in the Pact DSL, and the movie elements are defined as their type, with the element key as the method argument. `Id` and `Year` are defined as integer types. `name` is declared as a string type. Each type method is a Pact DSL matching method; if no example value is provided within the method, then a random value will be generated. Find more matching methods available in the Pact documentation (https://mng.bz/4pYV). The final part of the builder adds the interaction to the Pact contract, as shown in the following listing.

Listing 5.12 Pact Builder, part 3

```
.toPact(V4Pact::class.java)
```